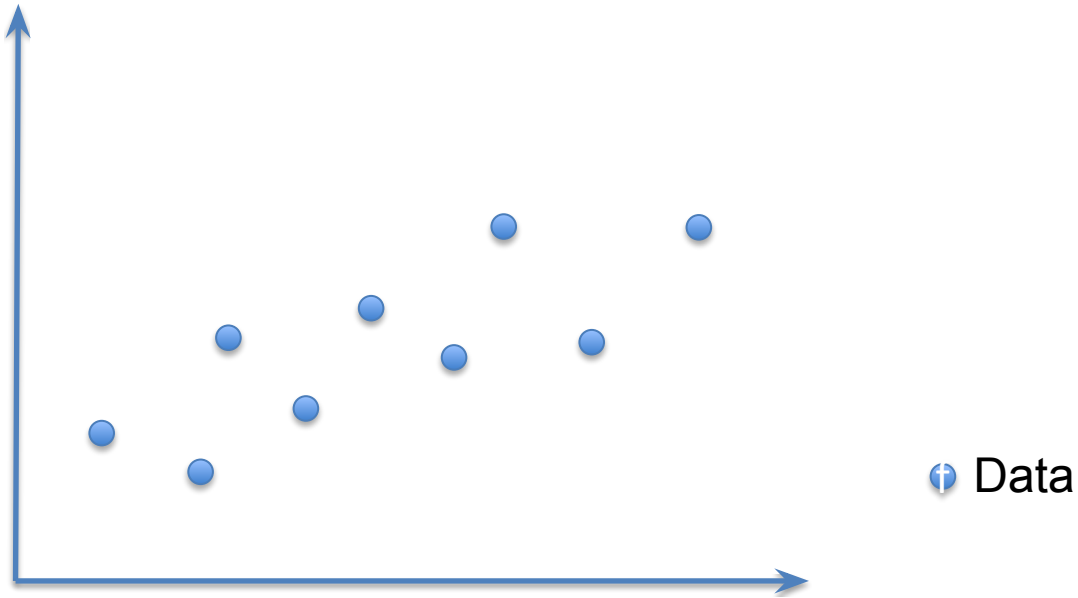
Técnicas Avanzadas de Data Mining y Sistemas Inteligentes

Maestría en Informática
Escuela de Posgrado
Pontificia Universidad Católica del Perú

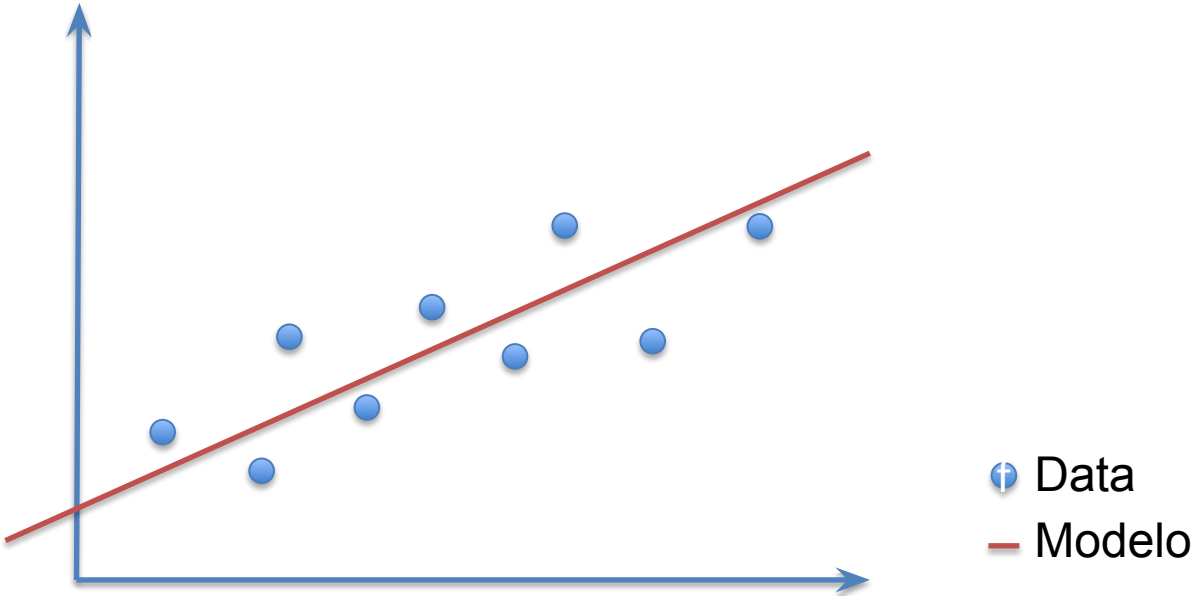
2021-2

Regresión Lineal Simple

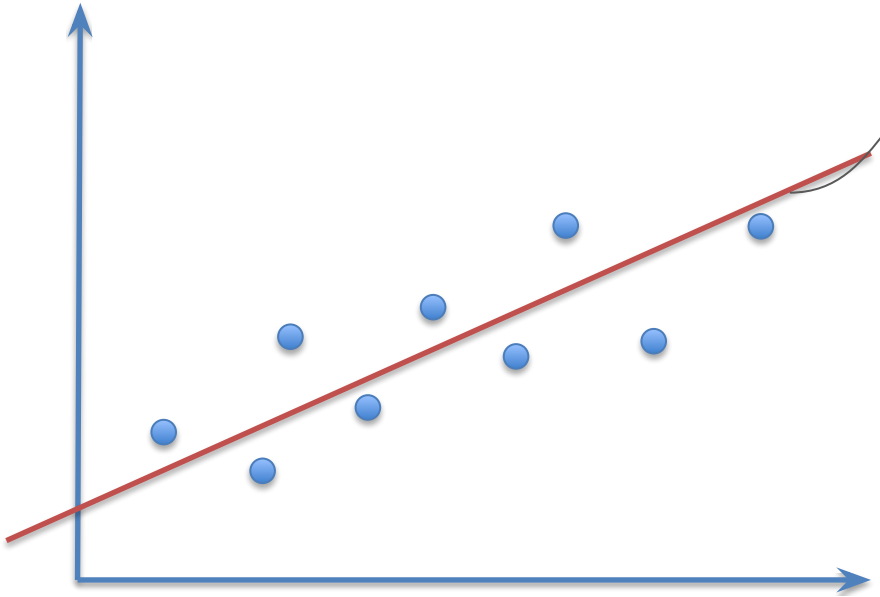
Regresión lineal simple



Regresión lineal simple



Regresión lineal simple



Data

Modelo

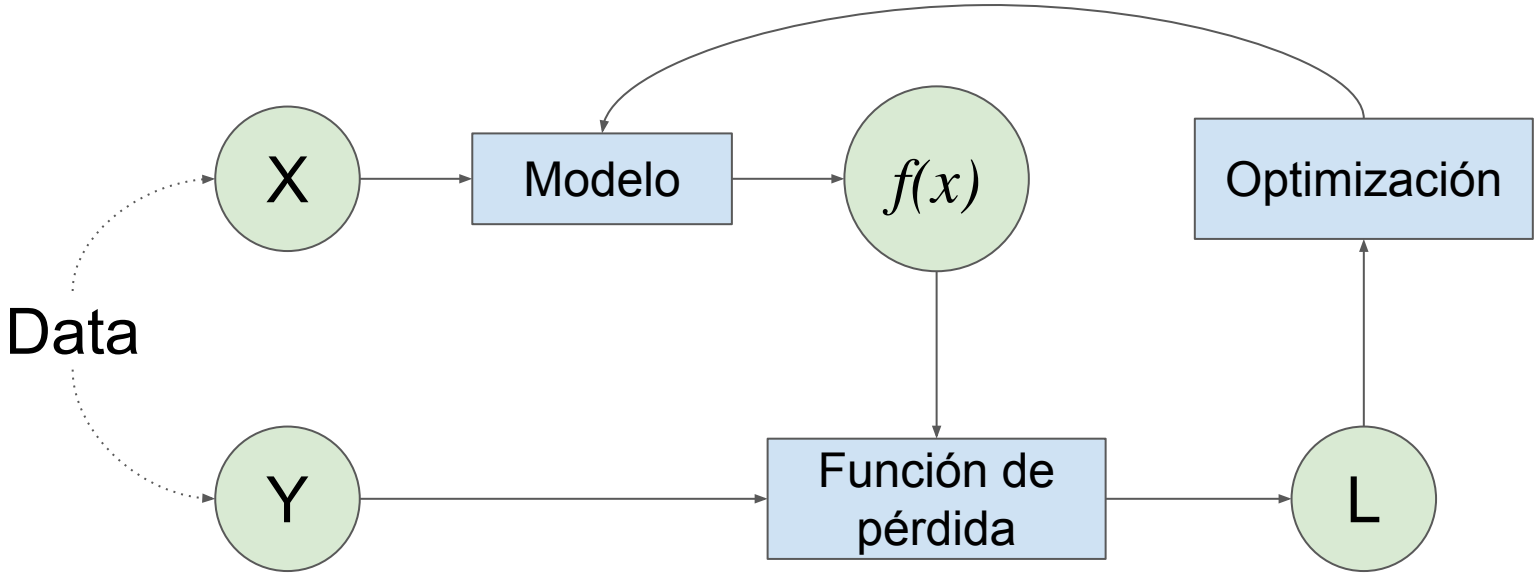
$$f(x) = ax_1 + bx_2 + cx_3 + d$$

Donde:

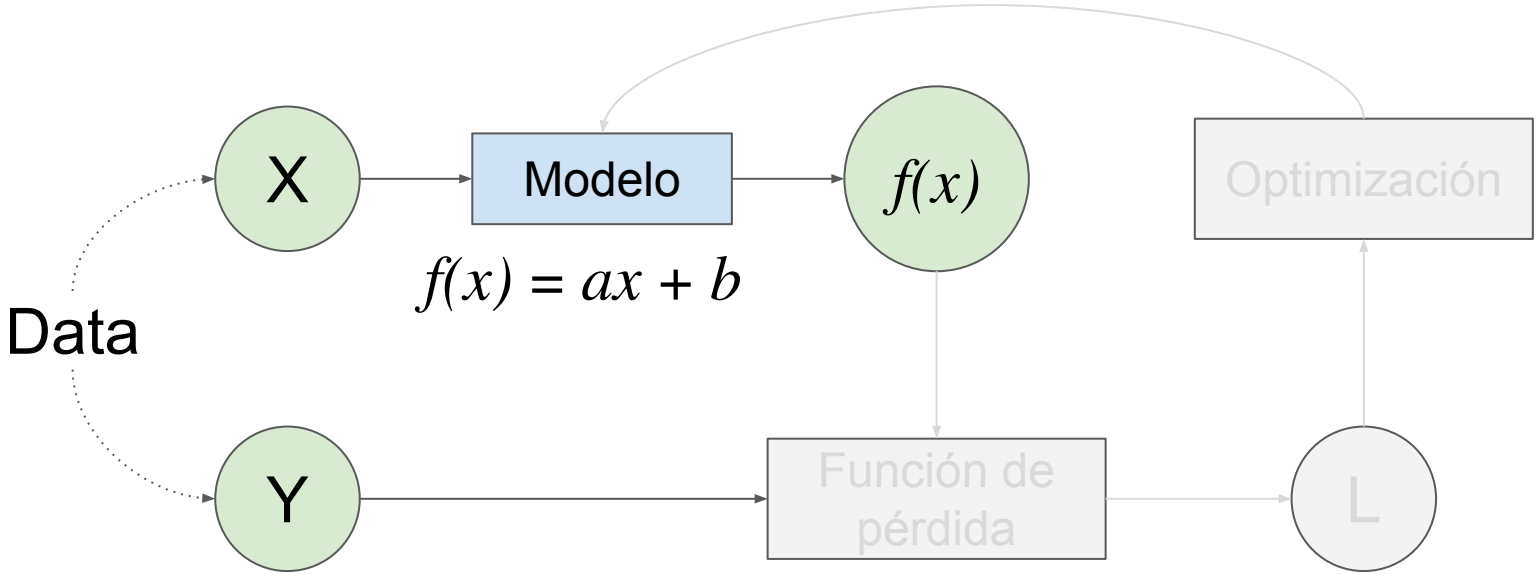
a = pendiente

b = término de intercepción (bias)

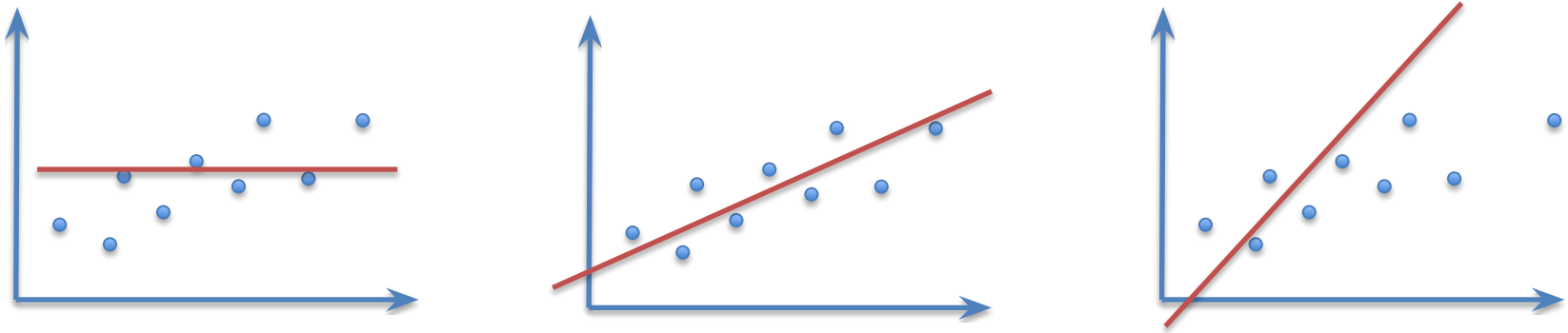
Regresión lineal simple



Regresión lineal simple

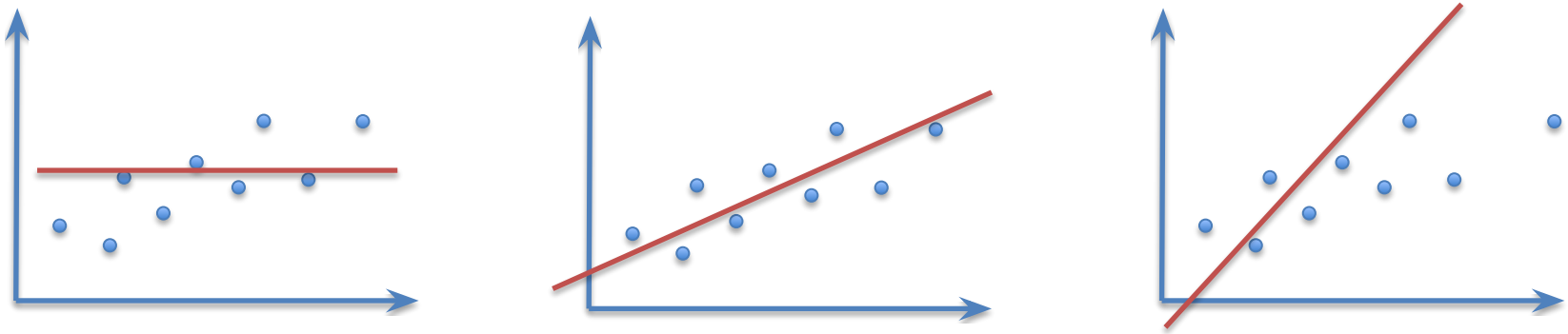


Función de pérdida



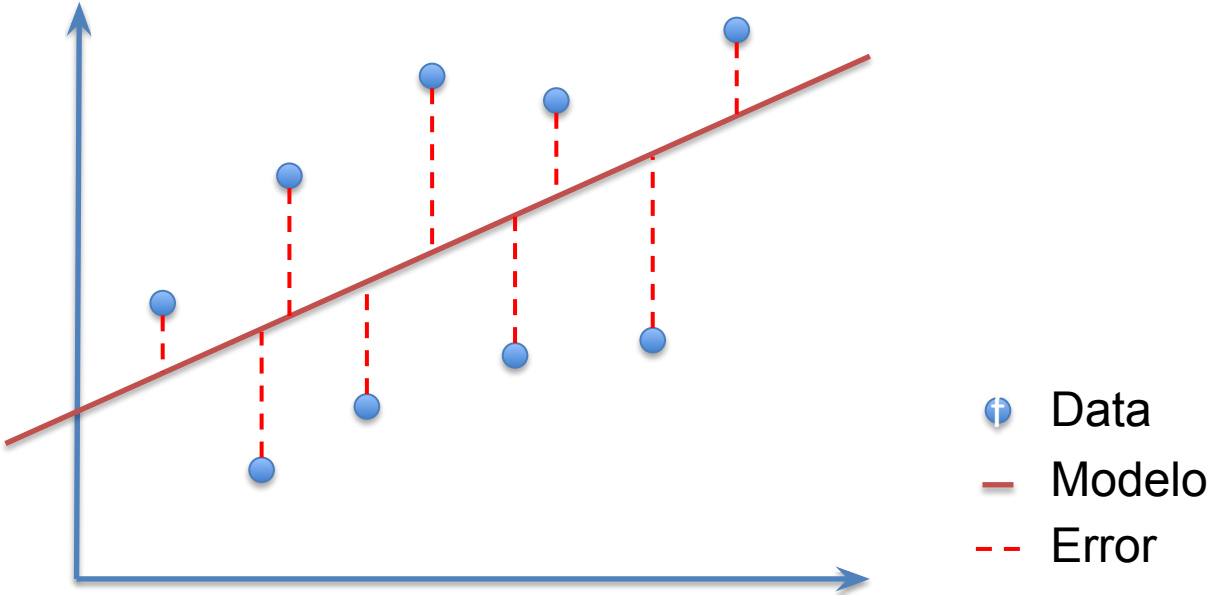
¿Cuál de los modelos describe mejor la data?

Función de pérdida

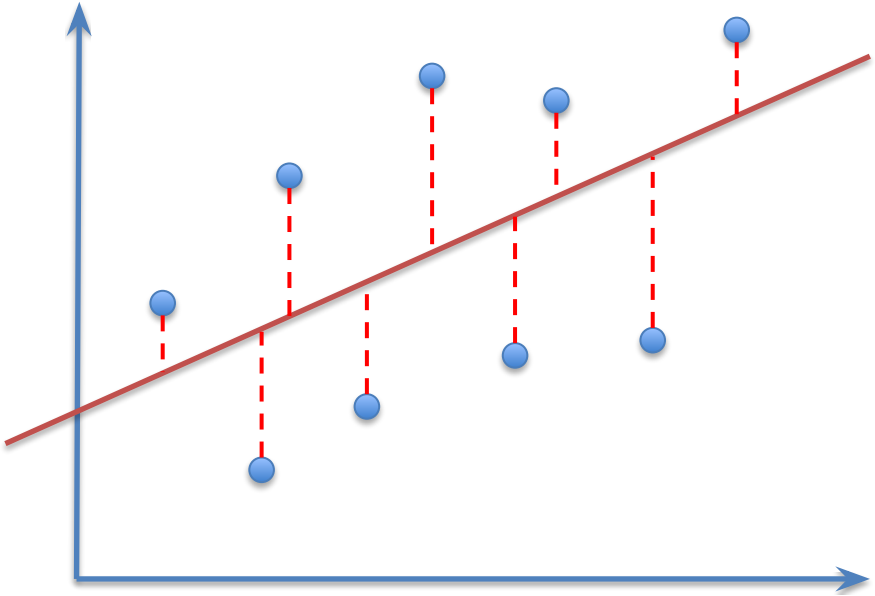


¿Cuál de los modelos describe mejor la data?
Evaluamos cada uno con una **función de pérdida** y escogemos el que tenga la menor pérdida.

Función de pérdida: MSE (Mean squared error)



Función de pérdida: MSE (Mean squared error)



Mean squared error:

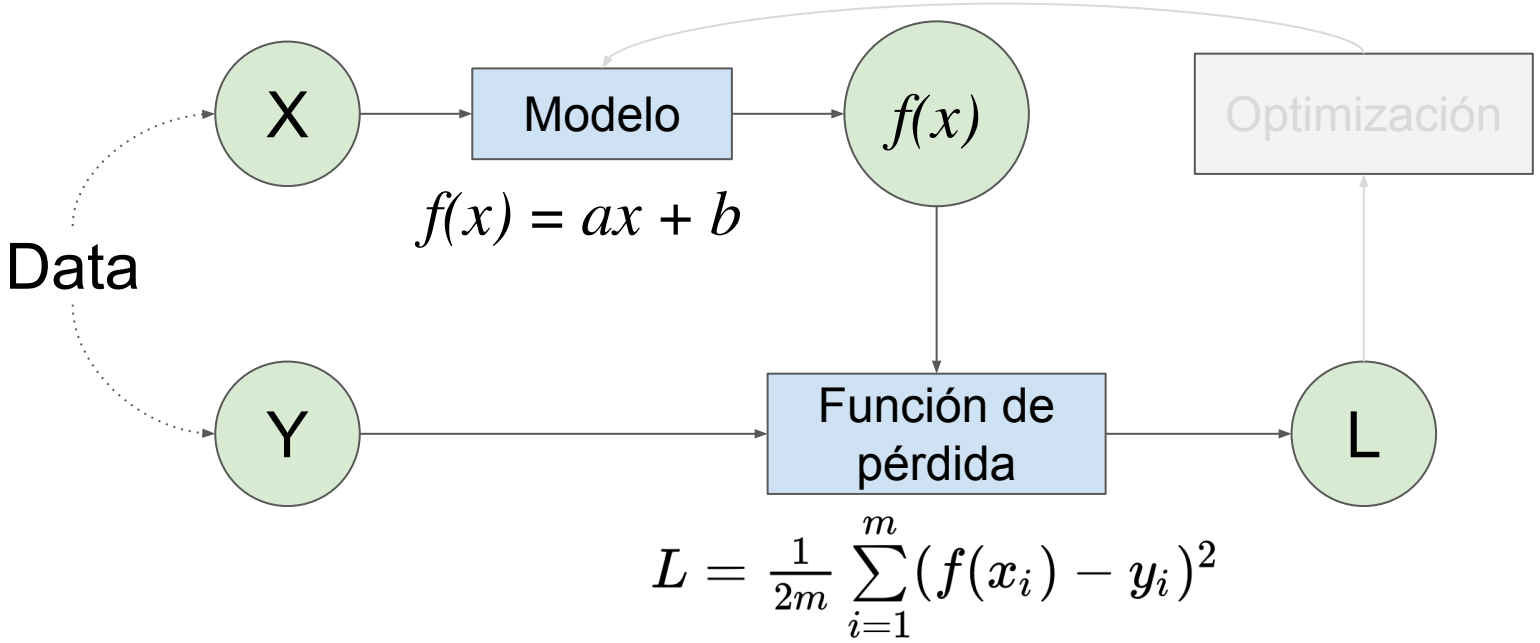
$$L = \frac{1}{2m} \sum_{i=1}^{m} (f(x_i) - y_i)^2$$

Data x

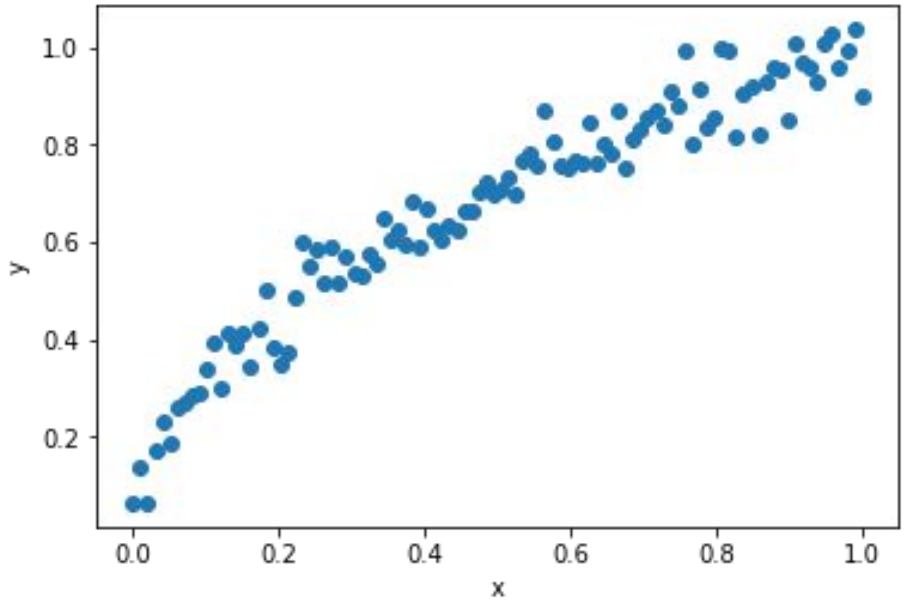
Modelo f(x)

Error f(x) - y

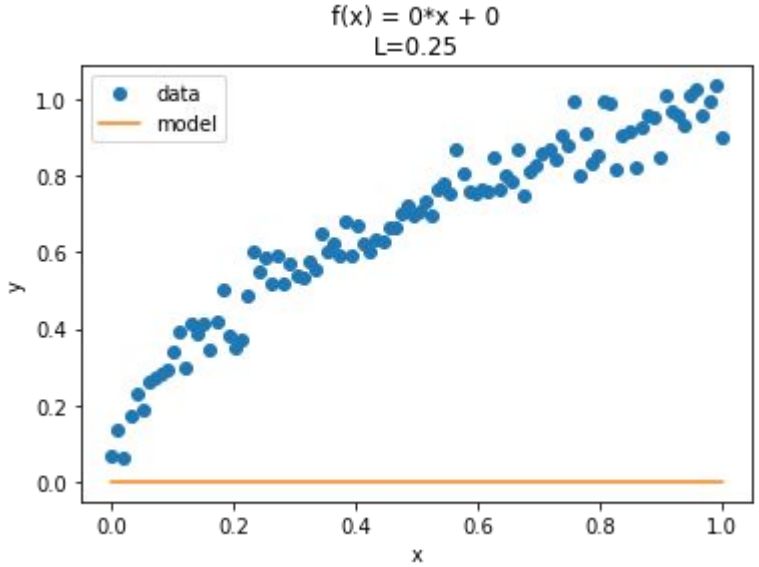
Función de pérdida: MSE (Mean squared error)



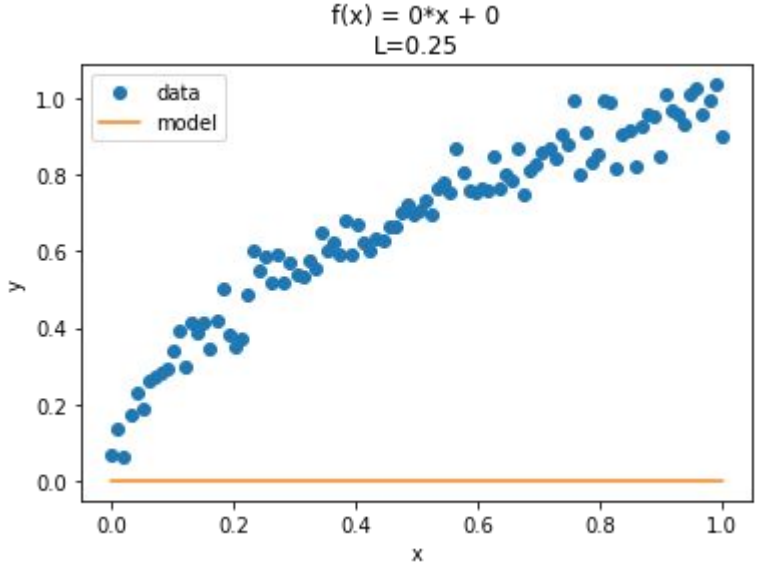
Optimización: Gradient descent

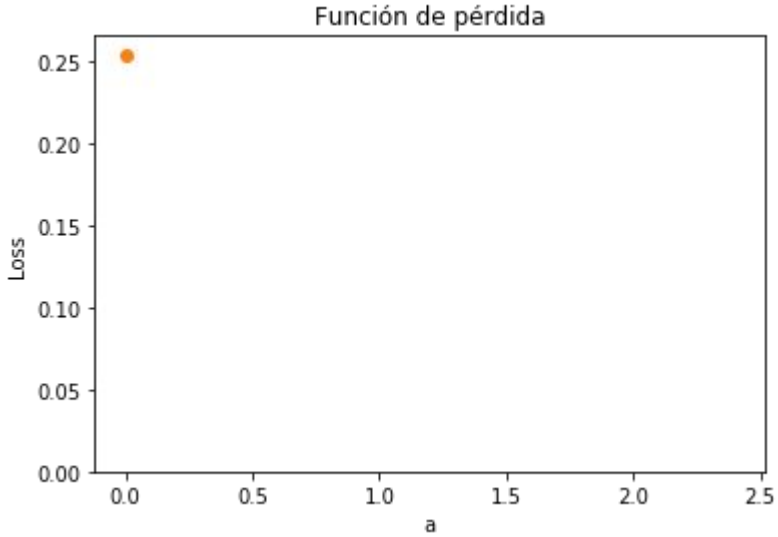


Optimización: Gradient descent

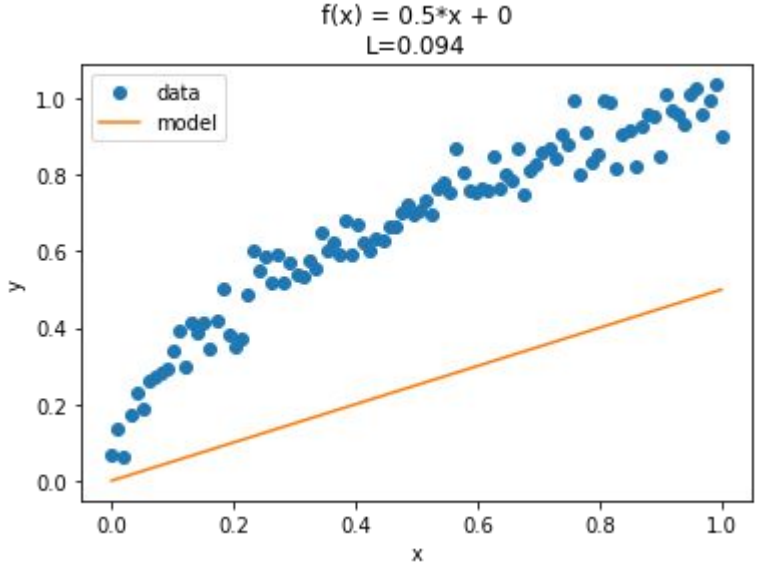


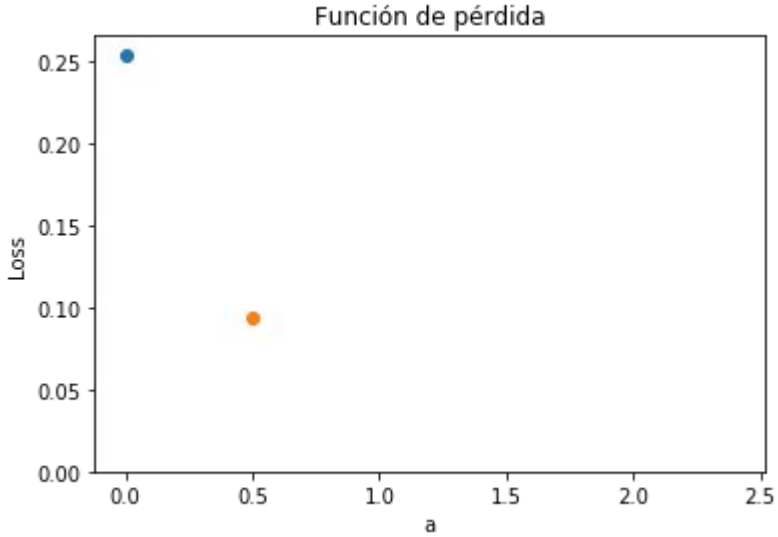
Optimización: Gradient descent



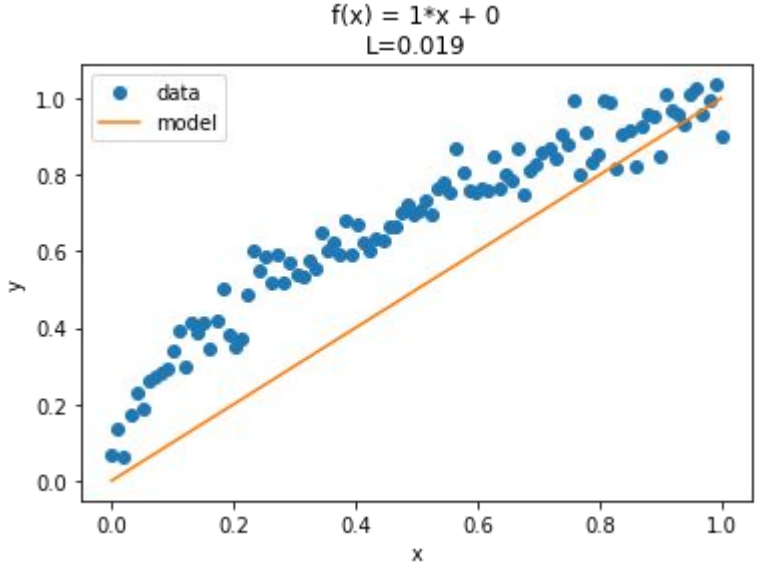


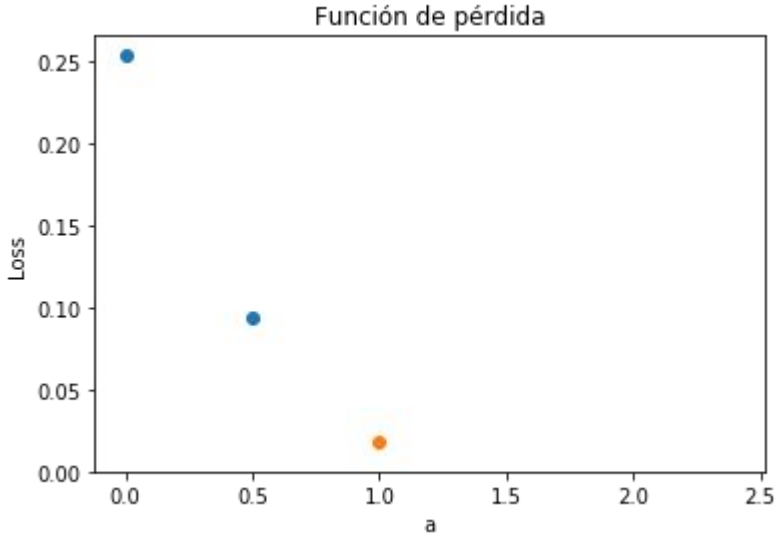
Optimización: Gradient descent



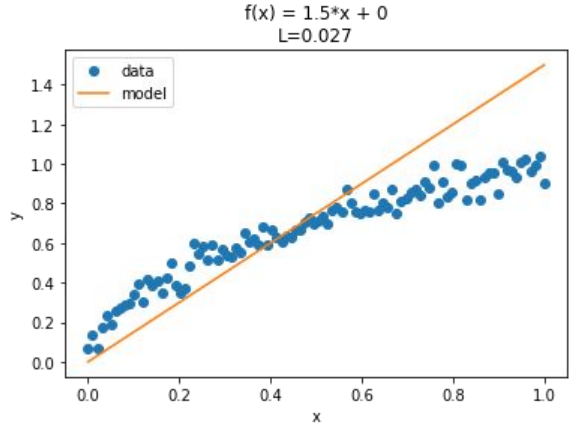


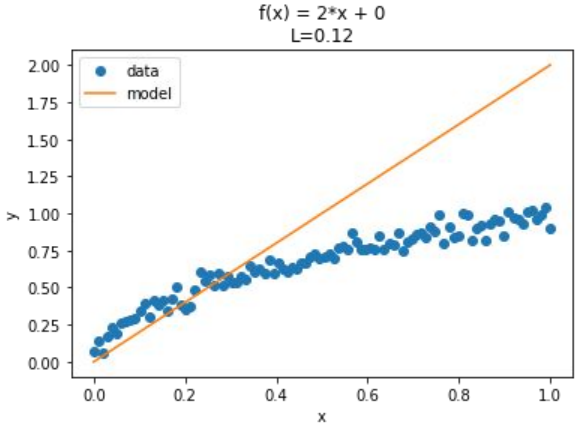
Optimización: Gradient descent

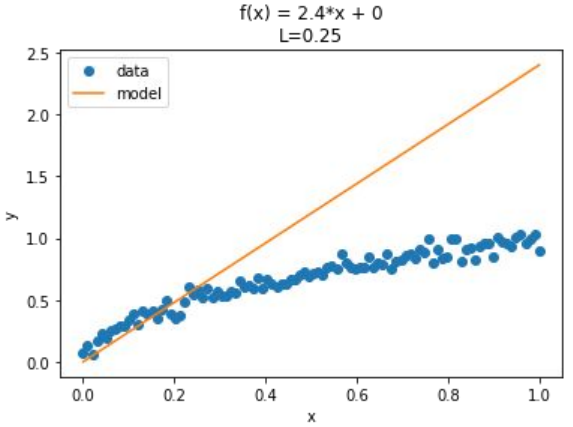


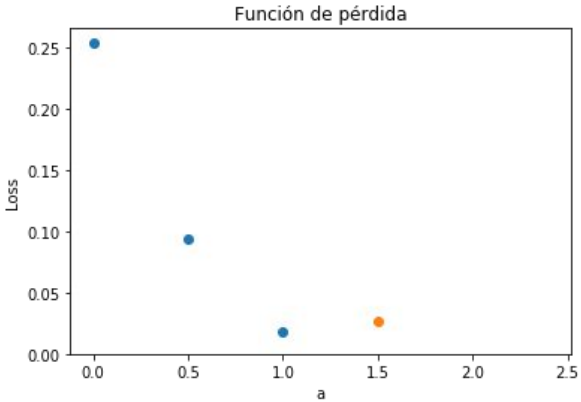


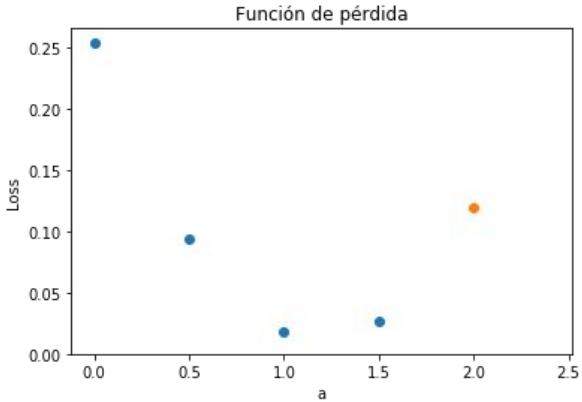
Optimización: Gradient descent

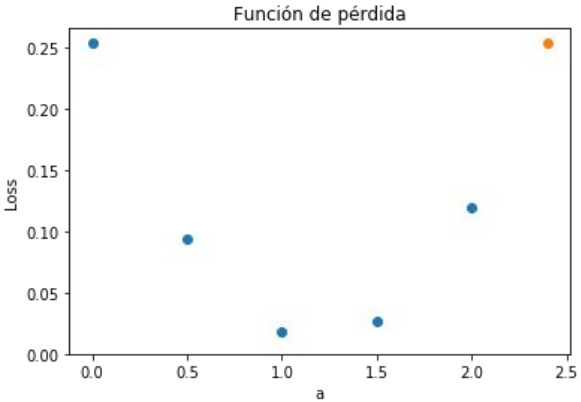




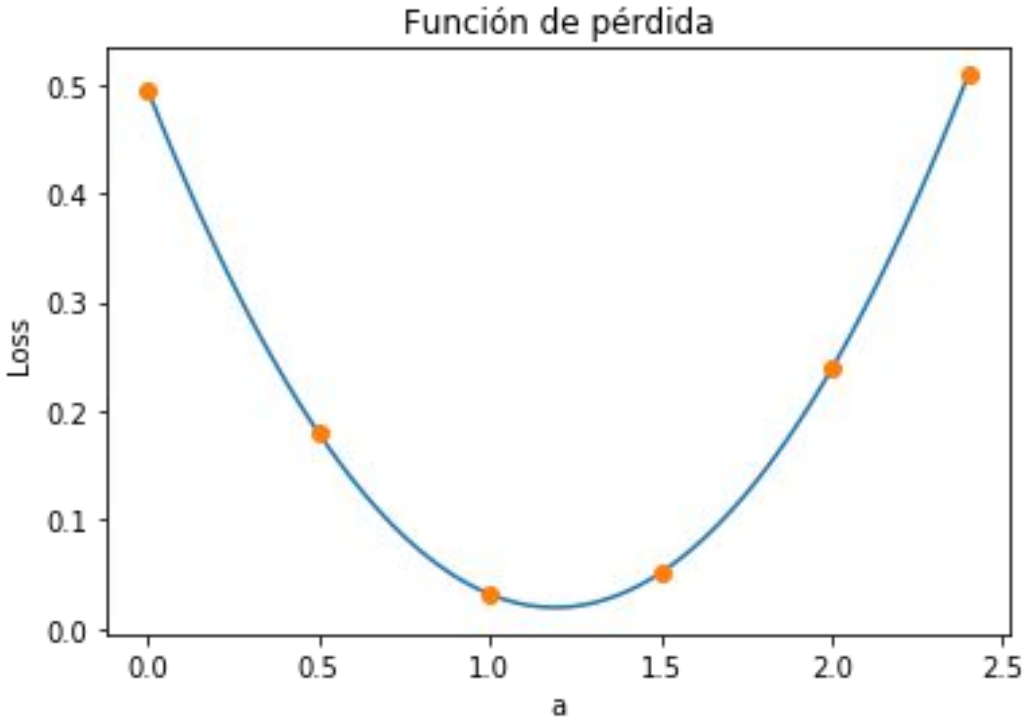




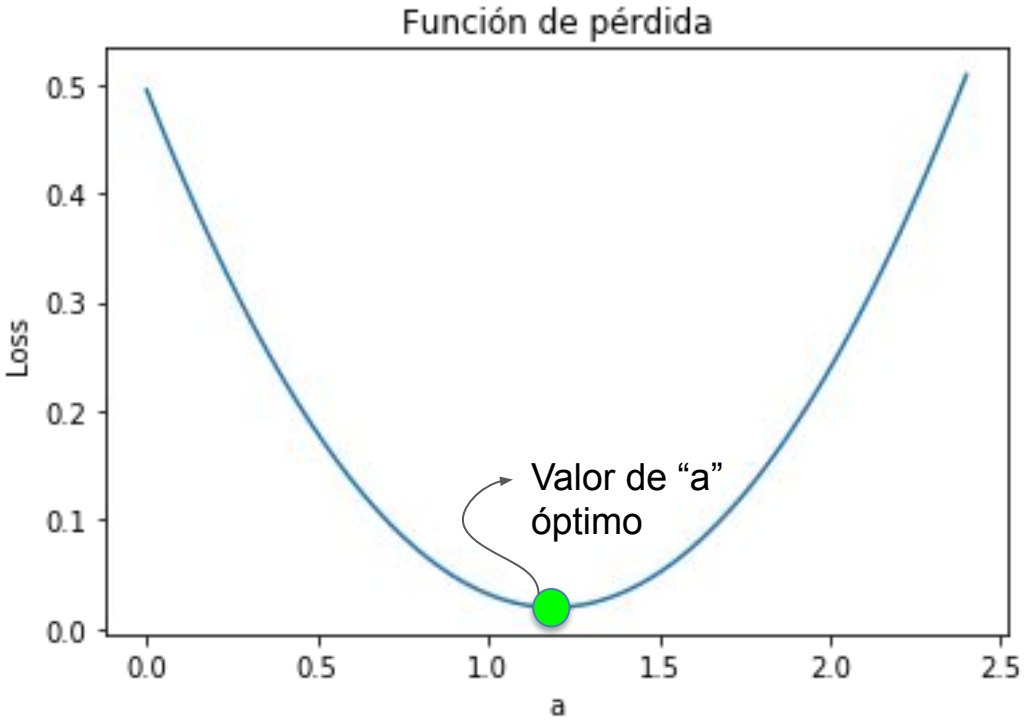




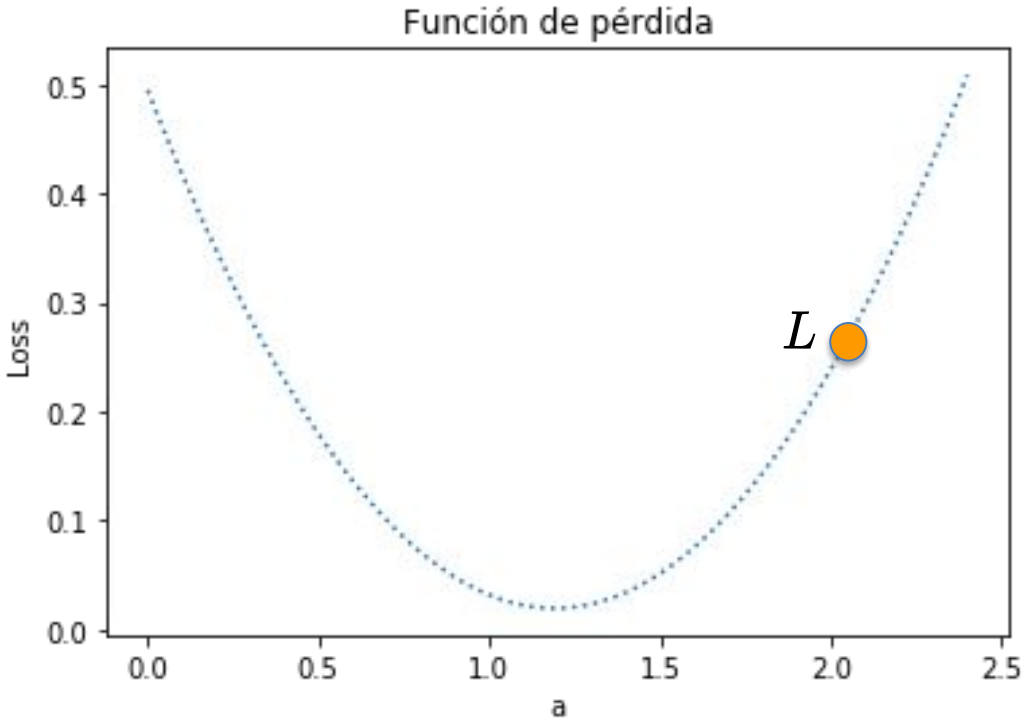
Optimización: Gradient descent



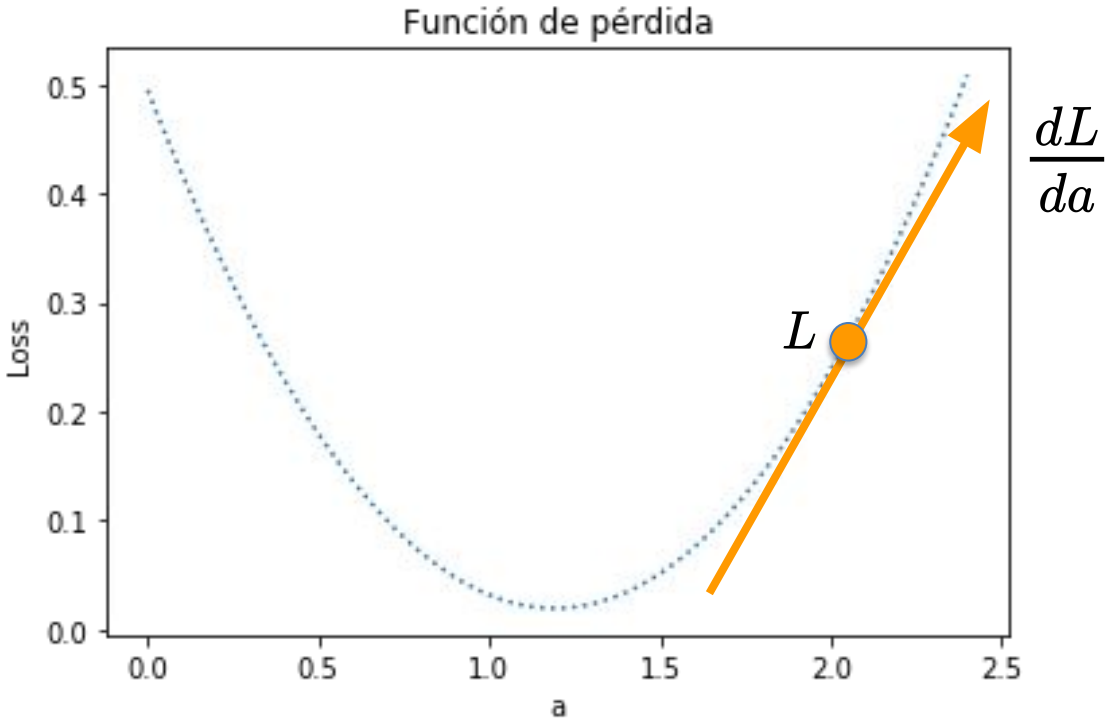
Optimización: Gradient descent



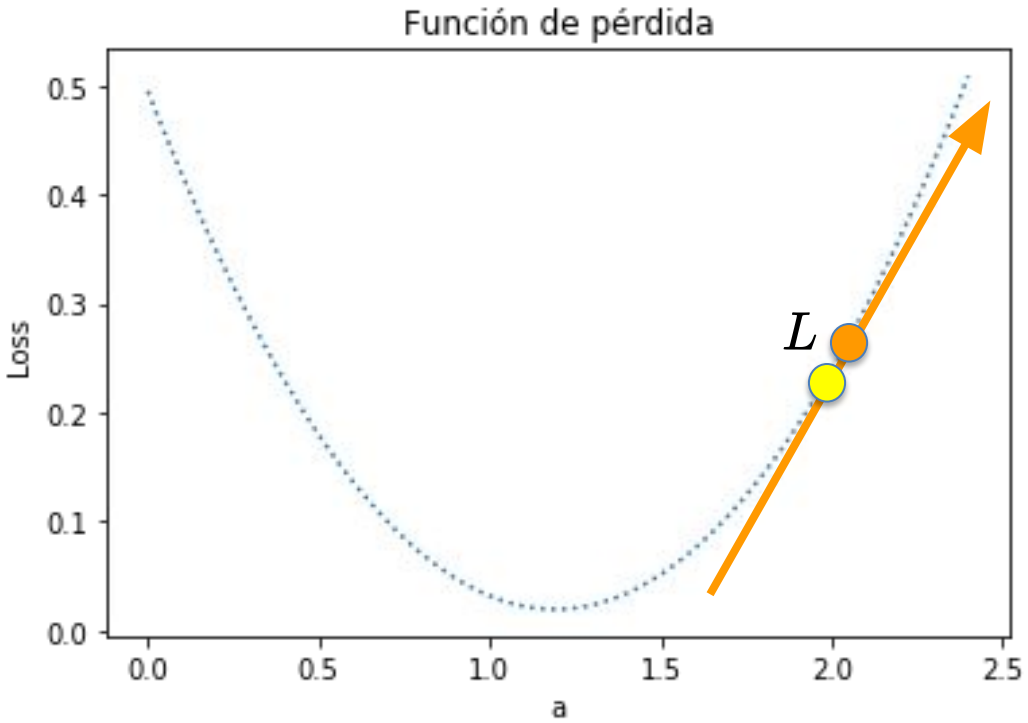
Optimización: Gradient descent



Optimización: Gradient descent



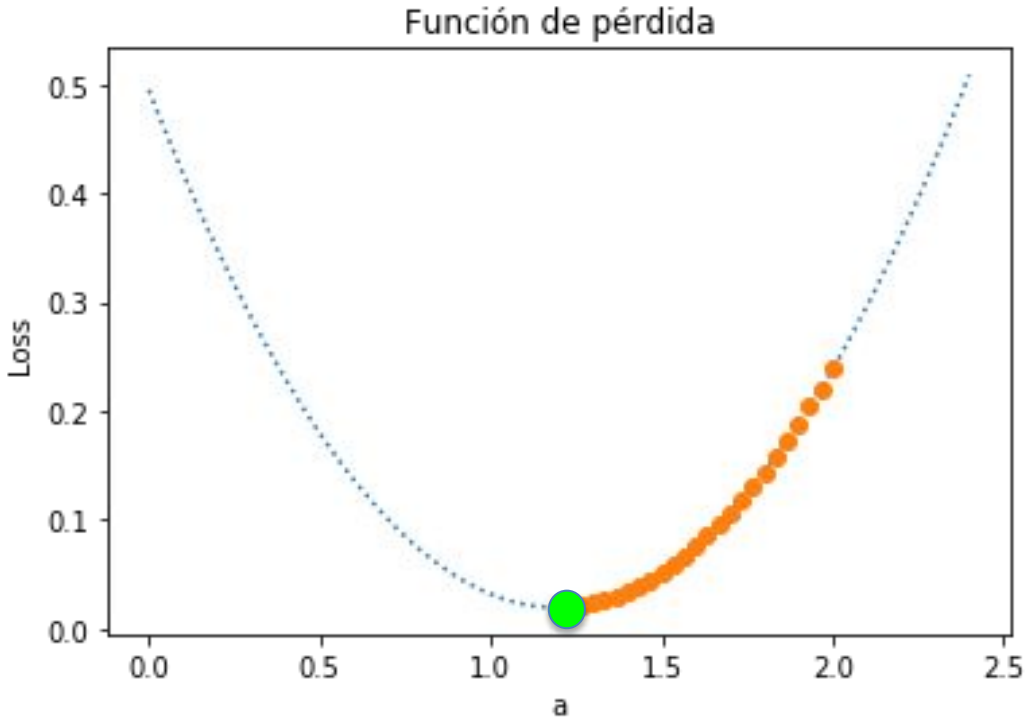
Optimización: Gradient descent



$$\frac{dL}{da}$$

$$new\,a = a - lr * (\frac{dL}{da})$$

Optimización: Gradient descent

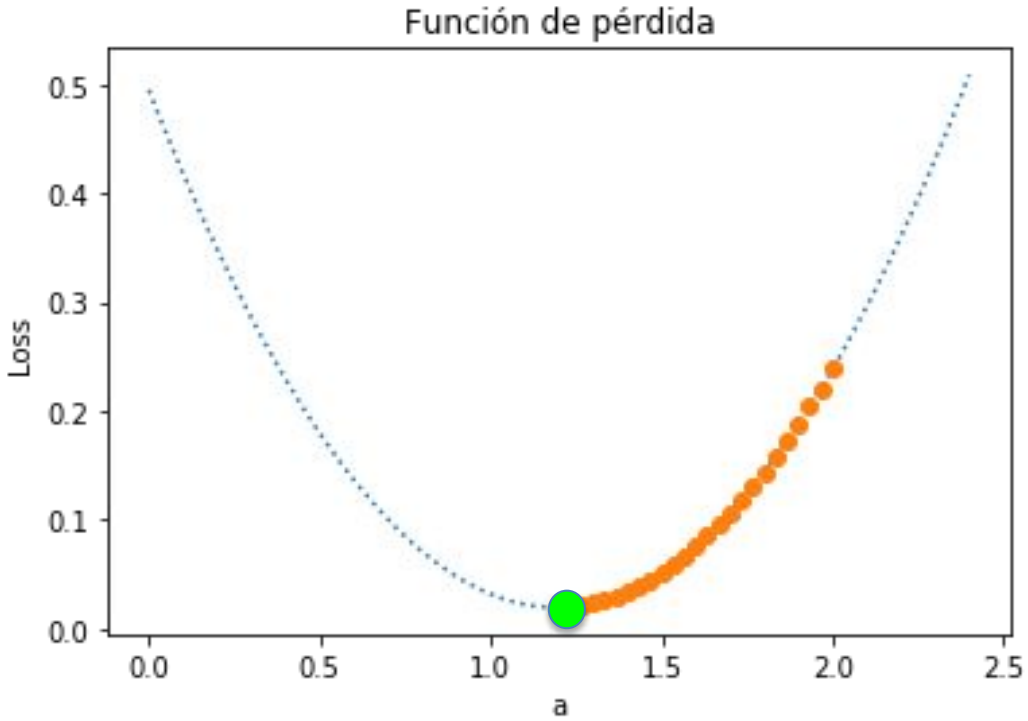


$$\frac{dL}{da}$$

$$new\,a = a - lr * (\frac{dL}{da})$$

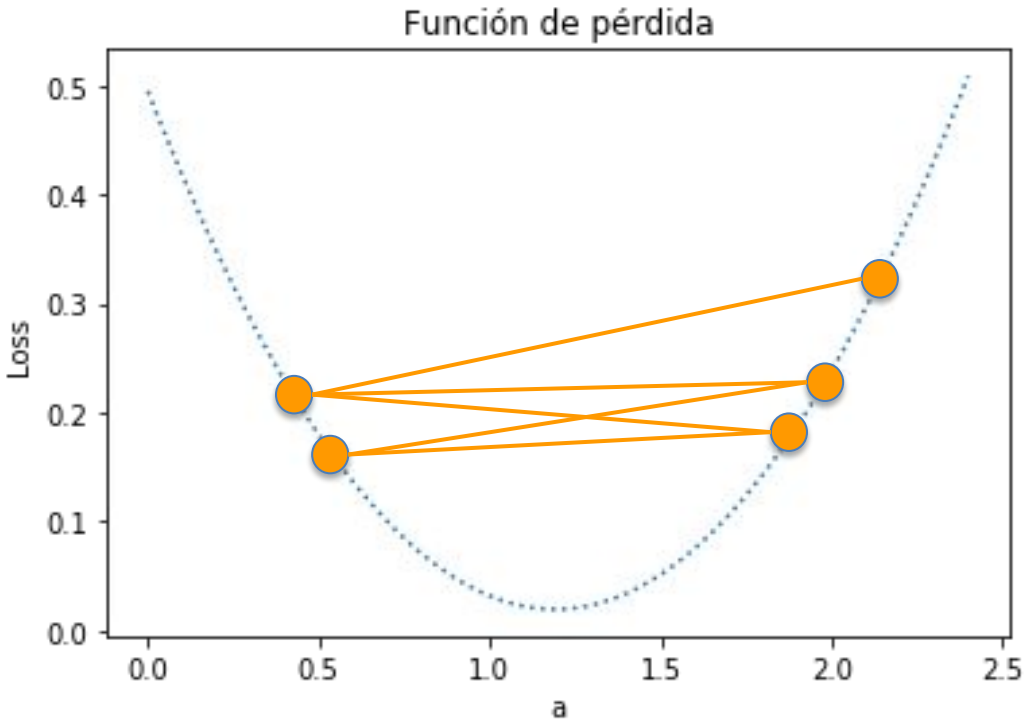
Se itera la optimización hasta converger.

Learning rate: lr



Con un lr bajo, el proceso de optimización puede ser largo.

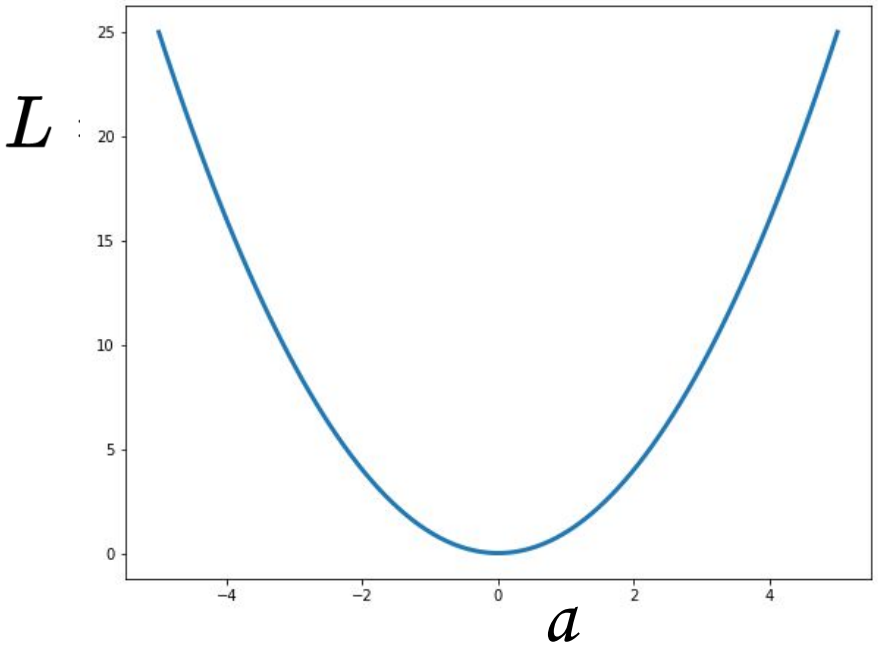
Learning rate: lr

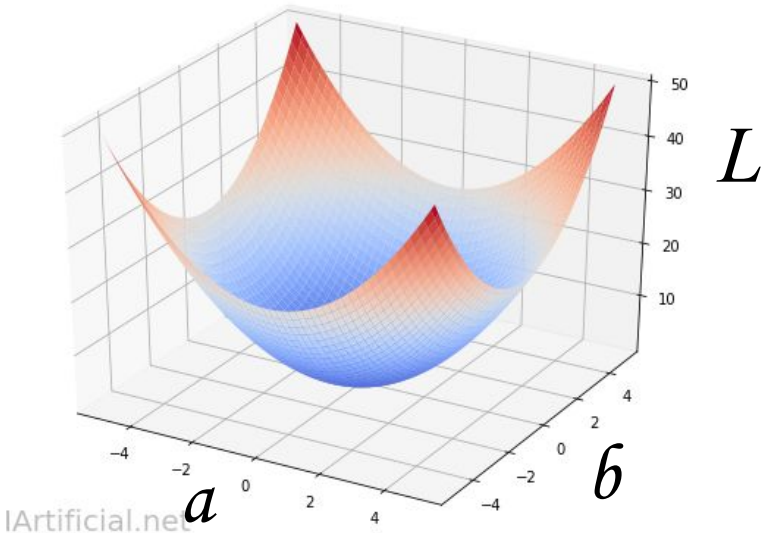


Pero con un lr muy alto, llegar al punto óptimo se puede volver imposible.

Optimización: Gradient descent

$$f(x_i) = ax_i + b$$





Optimización: Gradient descent

$$f(x_i) = ax_i + b$$

$$L = \frac{1}{2m}\sum_{i=1}^{m}(f(x_i) - y_i)^2$$

$$\frac{dL}{da} = \frac{d(\frac{1}{2m}\sum_{i=1}^{m}(f(x_i)-y_i)^2)}{da} = \frac{1}{2m}\sum_{i=1}^{m}(2(f(x_i) - y_i)(\frac{d(ax_i+b)}{da})) = \sum_{i=1}^{m}(f(x_i) - y_i)(x_i)$$

Optimización: Gradient descent

$$f(x_i) = ax_i + b$$

$$L = \frac{1}{2m}\sum_{i=1}^{m}(f(x_i) - y_i)^2$$

$$\frac{dL}{da} = \frac{d(\frac{1}{2m}\sum_{i=1}^{m}(f(x_i)-y_i)^2)}{da} = \frac{1}{2m}\sum_{i=1}^{m}(2(f(x_i) - y_i)(\frac{d(ax_i+b)}{da})) = \sum_{i=1}^{m}(f(x_i) - y_i)(x_i)$$

$$\frac{dL}{db} = \frac{d(\frac{1}{2m}\sum_{i=1}^{m}(f(x_i)-y_i)^2)}{db} = \frac{1}{2m}\sum_{i=1}^{m}(2(f(x_i) - y_i)(\frac{d(ax_i+b)}{db})) = \sum_{i=1}^{m}(f(x_i) - y_i)$$

Optimización: Gradient descent

$$f(x_i) = ax_i + b$$

$$L = \frac{1}{2m}\sum_{i=1}^{m}(f(x_i) - y_i)^2$$

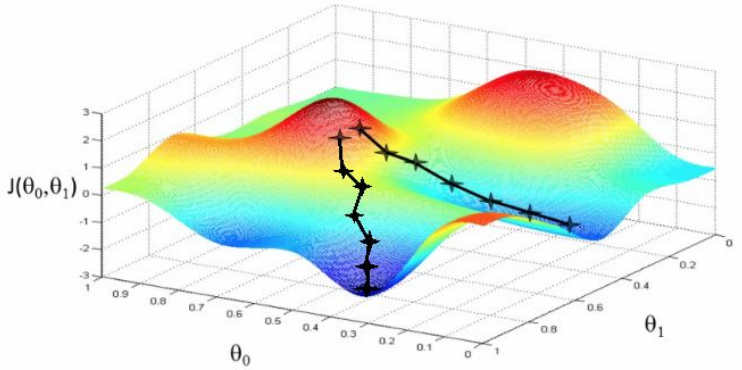
$$\frac{dL}{da} = \sum_{i=1}^{m}(f(x_i) - y_i)(x_i)$$

$$\frac{dL}{db} = \sum_{i=1}^{m}(f(x_i) - y_i)$$

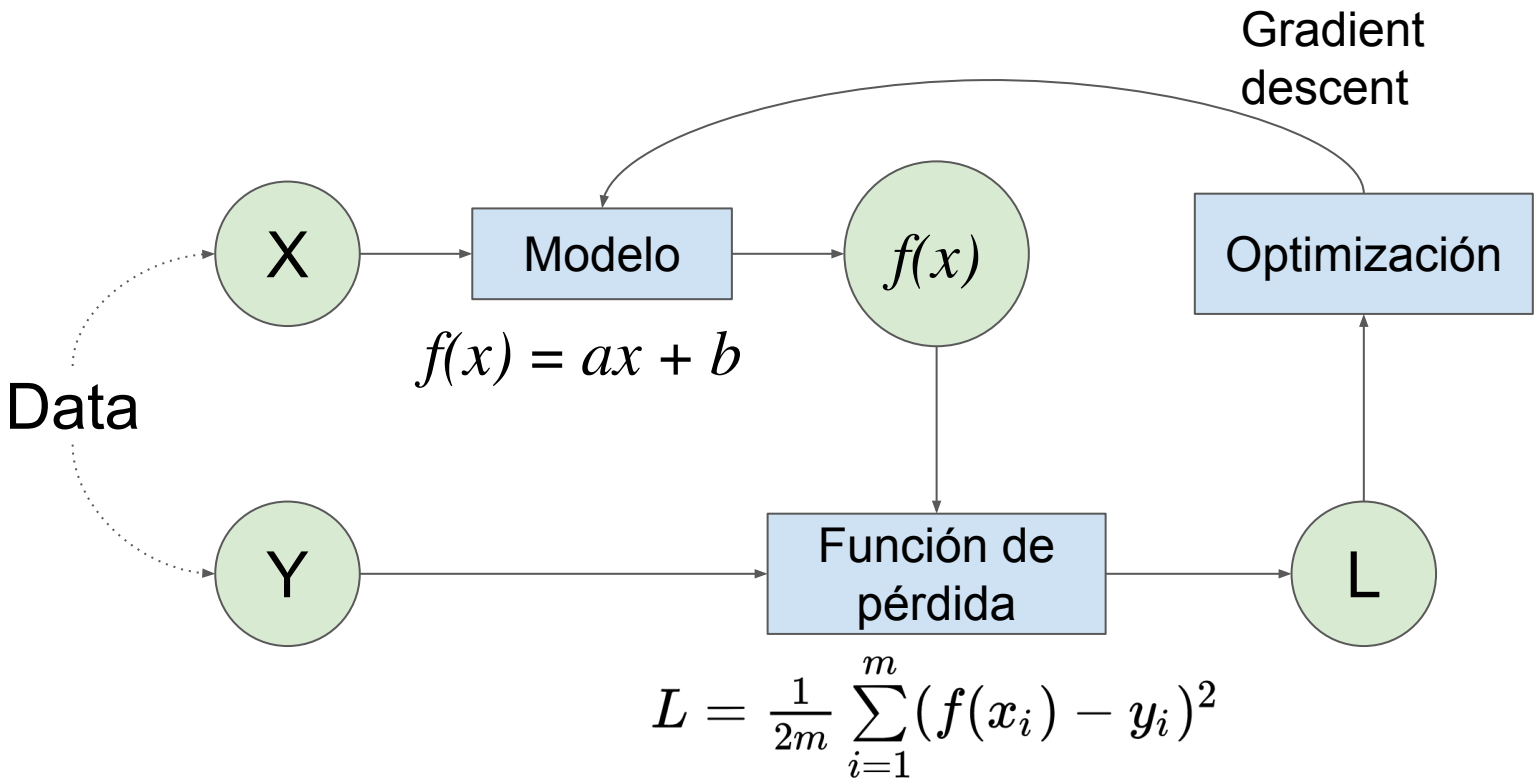
$$iterar:$$

$$new\, a = a - lr * (\frac{dL}{da})$$

$$new\, b = b - lr * (\frac{dL}{db})$$



Optimización: Gradient descent



Código

Herramientas

Ambiente de trabajo







ANACONDA

colab

Librerías

pandas

$$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$$









TensorFlow

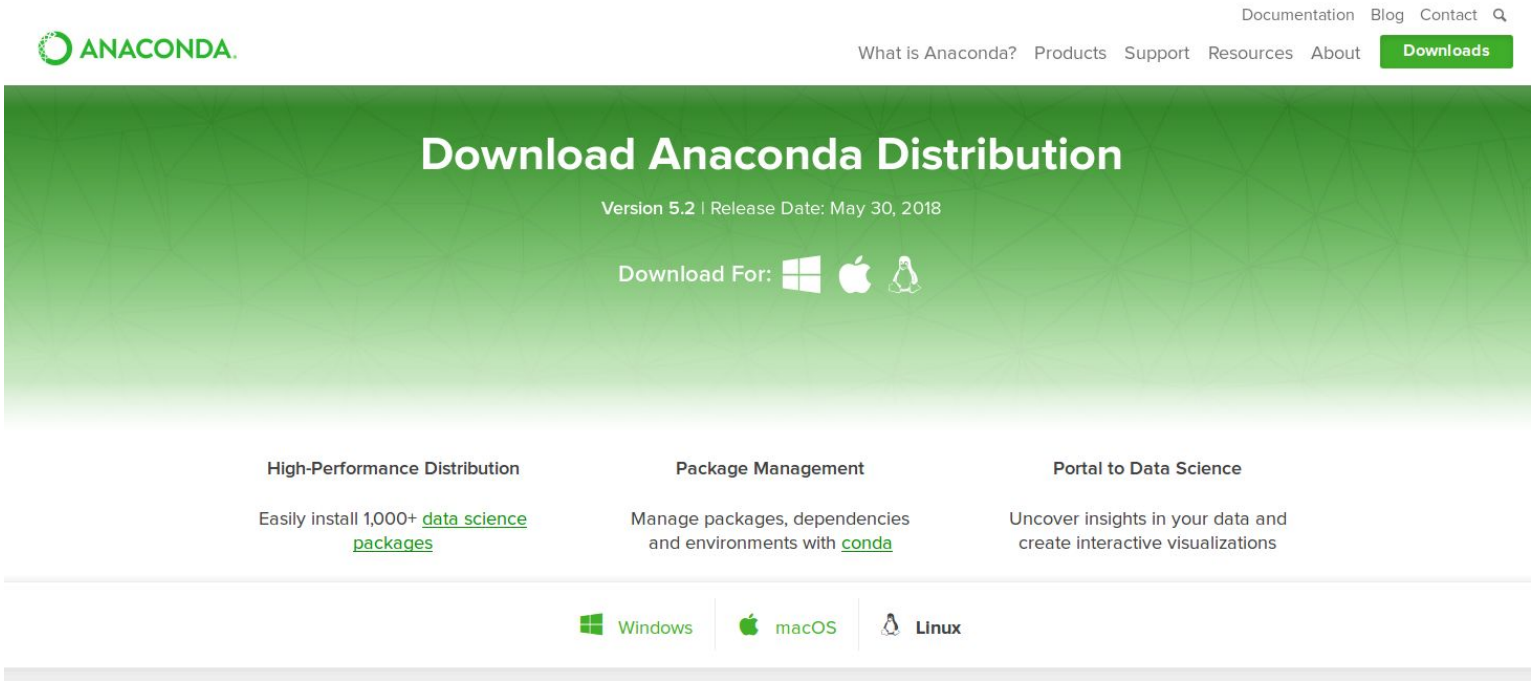




PyTorch

Anaconda (package manager)

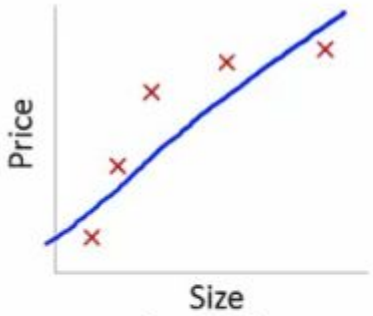
https://www.anaconda.com/download



Notebook

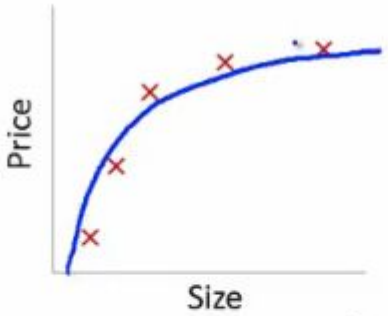
Validación de modelos

Overfitting



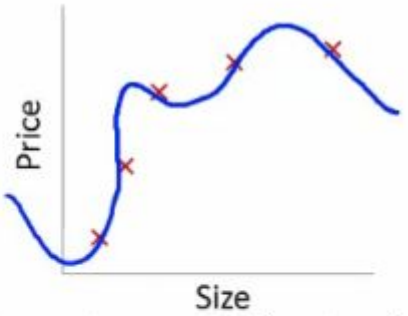
$\theta_0 + \theta_1 x$

High bias
(underfit)



$\theta_0 + \theta_1 x + \theta_2 x^2$

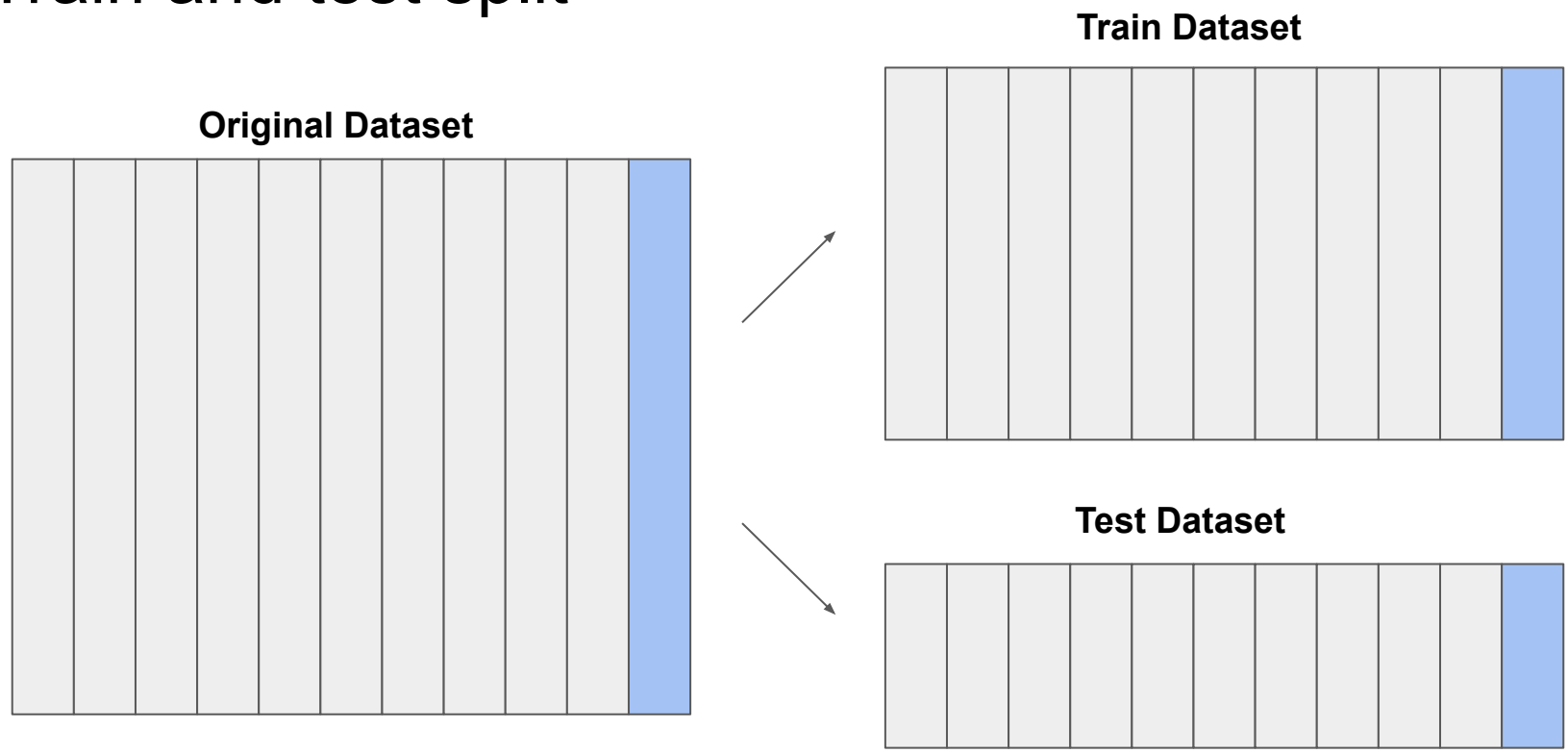
"Just right"



$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

High variance
(overfit)

Train and test split



Even better: Train, validation and test split

